Foundations of Computational Linguistics - Lecture 4: Neural architectures for sentence processing (27 Oct)

Lecture overview

Lecture 4 explores the evolution of neural architectures for sentence processing, beginning with foundational feed-forward networks where neurons function as logistic regression units stacked into multi-layer perceptrons. The session details how word embeddings map vocabulary into dense vector spaces where semantic similarity corresponds to vector proximity, enabling tasks like language modeling and named entity recognition through context windows. To handle variable-length sequences, the lecture contrasts the Continuous Bag of Words (CBOW) approach, which sums embeddings but loses ordering information, with Recurrent Neural Networks (RNNs) that maintain temporal state. This progression leads to the introduction of attention mechanisms, which resolve RNN bottlenecks by computing weighted averages of representations based on relevance, eventually allowing attention to replace recurrence entirely in modern architectures.

The core of the lecture focuses on the Transformer architecture, which utilizes self-attention to allow tokens to attend to all other tokens simultaneously, capturing bidirectional context and resolving polysemy more effectively than static embeddings or RNNs. Specific implementations like BERT leverage this through masked language modeling, where the model predicts missing tokens using surrounding context, followed by fine-tuning for specific tasks using a special <CLS> token for sentence representation. In contrast, generative models like GPT employ causal attention to restrict access to future tokens, enabling autoregressive next-token prediction. The lecture concludes by emphasizing the power of scale, noting that training these architectures on vast corpora for the single objective of word prediction allows them to implicitly master diverse downstream tasks such as sentiment analysis, question answering, and summarization without explicit task-specific training.

Slide 1: faculty of arts

This lecture introduces the fourth session on neural architectures for sentence processing within the Faculty of Arts' Computational Linguistics course. Delivered by Tim Van de Cruys on Tuesday, October 20, 2026, the session focuses on how neural models handle linguistic structures. No additional clarifications or examples were provided in the aligned transcript for this introductory slide.

Slide 2: Reminder

A neuron functions as a logistic regression unit, processing inputs through weights and a bias to produce an output via an activation function. By feeding an input vector into multiple such units, we generate an output vector that can serve as the input for subsequent layers of logistic regression functions. This stacking mechanism forms the basis of multi-layer neural architectures, allowing complex transformations of data through sequential processing stages.

Slide 3: Feed-forward

This slide defines a feed-forward architecture as a multi-layer perceptron with two hidden layers, where the input x is transformed through weighted sums and activation functions to produce output y. The network's parameters (θ) consist of weights (W_1, W_2, W_3) and biases (b_1, b_2), which are learned during training. The use of multiple hidden layers in this structure is explicitly identified as the defining characteristic of deep learning.

Slide 4: Word embeddings

Word embeddings represent each word i as a small, dense vector $v_i \in \mathbb{R}^d$, where the dimension d typically ranges from 50 to 1000. These vectors form a matrix of size $V \times d$ (vocabulary by embedding size), effectively mapping words into a real-valued, low-dimensional space. A core property of this representation is that semantically similar words are positioned close together, resulting in similar vector embeddings.

Slide 5: Feed-forward network

A feed-forward network for language modeling predicts the next word in a sequence using one non-linear hidden layer and a softmax classification layer. During training, parameters (weights) are tuned to maximize the probability of the correct next word. By mastering this prediction task on vast data, the model implicitly learns to solve diverse NLP tasks like sentiment analysis, question answering, and summarization without explicit task-specific training.

Slide 6: Feed-forward network

This feed-forward network performs named entity recognition by predicting tags for each token based on a context window including the current, previous, and next words. The architecture utilizes one non-linear hidden layer followed by a softmax classification layer to optimize the probability of correct tags, such as identifying location boundaries. Inputs consist of standard non-contextualized token embeddings combined with position information, and optionally segment embeddings, to encode necessary contextual features for the task.

Slide 7: Feed-forward network

This is a feed-forward word-segmentation example associated with Pei et al. (2014). A feed-forward network maps a fixed input representation to a label or prediction through successive layers. The slide identifies the application and citation; details beyond the displayed setup should be taken from the cited paper.

Slide 8: Encoding sentences

To encode variable-length sentences for fixed-dimensional feed-forward networks, three architectures address the challenge of representing sequences: Continuous Bag of Words (CBOW) sums embedding vectors but loses ordering information; Recurrent Neural Networks use connections to encode sentence history; and the Transformer architecture employs self-attention, allowing tokens to attend to all other tokens for contextualization. While CBOW is simple, Transformers are now standard in large language models because they effectively capture relationships across the entire sequence without relying on sequential processing.

Slide 9: CBOW: induction of embeddings

In the CBOW model, the current word is predicted from its surrounding context words by summing their corresponding embeddings. This prediction relies on calculating the pairwise sum of values for each feature across the context vectors. The provided transcript discusses sentence-level inference tasks like entailment and semantic similarity using class tokens, which are distinct from the specific word-prediction mechanism described in this slide's CBOW definition.

Slide 10: CBOW: sentiment classification

CBOW extends to sentiment classification by predicting sentiment from the words in a sentence or document. The model generates this prediction by summing the context embeddings, specifically performing a pairwise addition of corresponding embedding features. This summation approach allows the architecture to handle inputs with a variable number of words, such as the example phrase "i like this movie," to output a probability distribution for sentiments like positive.

Slide 11: Recurrent neural network

Unlike CBOW models which lack ordering and structure, Recurrent Neural Networks (RNNs) are designed to represent arbitrarily sized structured inputs as fixed-size vectors. This architecture specifically pays attention to the inherent structural properties of the data, enabling it to handle sequential information that non-structured models cannot process effectively.

Slide 12: Recurrent neural network

A recurrent neural network model processes sequential data by combining the current word input (x_1) with the previous hidden state (a_0) to generate the current hidden layer (a_1) via weight matrices U and W, followed by a sigmoid activation function. The final output probability distribution (y_1) is derived from this hidden state using weight matrix V and a softmax function. This architecture explicitly captures temporal dependencies through the recurrence of the hidden layer across timesteps.

Slide 13: Recurrent neural network

This slide illustrates a recurrent neural network architecture designed for language modeling, where the current input word is combined with the previous context to generate a prediction. The model utilizes word embeddings and weight matrices (W, U, V) to process the current word at time t alongside the hidden state from time $t-1$, ultimately producing a distribution over possible next words. This structure enables the network to maintain temporal dependencies by feeding prior contextual information forward through the sequence.

Slide 14: Recurrent neural network

Slide 14 introduces the recurrent neural network (RNN) architecture specifically applied to language modeling. The visual representation depicts a sequence of inputs ("I", "am", "think", "would") processed through hidden states (a_0 to a_m) using weight matrices U, W, and V to generate outputs (y). This structure illustrates how RNNs maintain temporal dependencies by passing information from previous time steps to subsequent ones within a sequence.

Slide 15: Recurrent neural network

Slide 15 introduces the Recurrent Neural Network (RNN) as a language model architecture. The visual representation depicts the network's internal state transitions, showing how inputs like "starting," "watching," and "not" are processed through weight matrices (W, U) to generate outputs such as "am" and "I." This diagram illustrates the core mechanism where previous activations influence current predictions within a sequence.

Slide 16: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) as a language model architecture, illustrated by the sequential processing of the sentence "I am watching a television football." The diagram depicts how the network maintains internal states across time steps to handle variable-length input sequences, a core capability for modeling linguistic context in computational linguistics.

Slide 17: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) as a language model architecture, illustrated by the sequence "I am watching television." The diagram depicts the network processing tokens like "television," "and," and "but" alongside a special "<stop>" token to manage sentence boundaries. This visual representation demonstrates how RNNs handle sequential data in computational linguistics by maintaining context across time steps for tasks such as text generation or prediction.

Slide 18: Recurrent neural network

A recurrent neural network functions as a language model by acting as a sentence encoder. The architecture processes input sequences through hidden states, where the current state a_1 is computed from the previous state a_0 and the current input I using weight matrices W and U. This mechanism allows the network to maintain context across time steps, effectively encoding entire sentences into a representation suitable for language modeling tasks.

Slide 19: Recurrent neural network

A recurrent neural network functions as a language model that acts as a sentence encoder. This architecture processes sequential input, such as the word sequence "I am I," by maintaining internal states across time steps to capture contextual dependencies within the sentence structure.

Slide 20: Recurrent neural network

A recurrent neural network functions as a language model that acts as a sentence encoder. The architecture processes sequential input, such as the phrase "I am watching," by maintaining internal states across time steps to capture contextual dependencies within the sentence structure. This mechanism allows the network to encode entire sentences rather than treating words in isolation.

Slide 21: Recurrent neural network

A recurrent neural network functions as a language model that acts as a sentence encoder. The architecture processes sequential input, such as the phrase "I am watching television," by transforming individual words into vector representations through weight matrices (U and W) to capture contextual dependencies across the sequence.

Slide 22: Recurrent neural network

A recurrent neural network functions as a language model and sentence encoder that processes sequential input, such as the phrase "I am watching television." By applying weight matrices (W and U) across time steps, the architecture aggregates information to capture the entire sentence's meaning within a single large vector representation.

Slide 23: Recurrent neural network

Slide 23 illustrates an encoder-decoder recurrent neural network architecture performing sequence-to-sequence translation. The diagram depicts the French input sequence "je le la television" being processed through weight matrices (W, U, V) and biases (b0, b1) to generate the English output "I am watching television," demonstrating how RNNs map variable-length source sequences to target sequences in computational linguistics.

Slide 24: Recurrent neural network

Slide 24 introduces the recurrent neural network architecture, specifically highlighting an encoder-decoder configuration. The diagram illustrates the flow of input sequences (represented by French words like "je," "regarde," "cherche," and "viens") through weight matrices (U, V) and biases (b1, b2) to generate outputs. This visual representation demonstrates how recurrent networks process sequential data within a computational linguistics context.

Slide 25: Recurrent neural network

An encoder-decoder RNN reads an input sequence into hidden states and uses a decoder to produce an output sequence. The recurrent state carries information across positions, allowing the model to handle variable-length input and output.

Slide 26: Recurrent neural network

Slide 26 introduces the Recurrent Neural Network (RNN) within an encoder-decoder architecture, illustrating a sequence-to-sequence translation task. The diagram depicts input tokens ("je regarde la la montre télévision météo") being processed through weight matrices (U, V, W) and biases (b3, b4) to generate output representations (x, y). This visual example demonstrates how RNNs handle sequential data in computational linguistics by encoding source sentences into a context vector for decoding.

Slide 27: Recurrent neural network

This slide illustrates a recurrent neural network configured as an encoder-decoder architecture, featuring weight matrices (W, V, U) and biases (b4, b5) that process sequential inputs. The example demonstrates the model generating the French sentence "je regarde la télévision" ("I watch television") followed by conjunctions and a stop token, highlighting how RNNs handle variable-length sequences in computational linguistics tasks.

Slide 28: Recurrent neural network

Recurrent neural networks are trained as very deep networks with shared parameters using backpropagation through time. Supervision varies by architecture: acceptors rely on the final state, transducers generate an output for each input (e.g., language modeling), and encoder-decoder systems use one RNN to encode a sequence into a vector and another to decode it (e.g., machine translation).

Slide 29: Recurrent neural network

A bi-directional recurrent neural network processes an input sequence in both forward and backward directions using separate RNNs. The final representation is formed by concatenating the forward and backward states, enabling the model to capture information from both past history and future context simultaneously.

Slide 30: Concrete RNN architectures

The page compares concrete recurrent architectures. A simple RNN updates a hidden state at each time step; gated variants such as LSTM address the difficulty of carrying useful information over long sequences.

Slide 31: Concrete RNN architectures

Simple Recurrent Neural Networks (RNNs) are limited in practice because they can only remember a narrow context due to the vanishing gradient problem. To address this, Long Short-Term Memory (LSTM) networks utilize a complex architecture specifically designed to capture long-term dependencies that standard RNNs cannot retain.

Slide 32: Recurrence: difficulties

Recurrent Neural Networks (RNNs) process sentences sequentially, word by word, which creates significant challenges. Because information from the start of a sequence must persist through many processing steps, capturing long-distance dependencies becomes difficult in practice. Furthermore, this sequential nature prevents parallel computation, as each step must wait for the previous one to complete before proceeding.

Slide 33: Bottleneck

In standard encoder-decoder architectures, the "bottleneck" problem arises because the entire meaning of a sentence is compressed into a single fixed-length vector, regardless of input length. This raises questions about whether a uniform vector size can adequately represent sentences of vastly different lengths, such as two-word versus thirty-word inputs. The proposed solution is to replace this single vector with an attention mechanism that allows each decoder prediction to dynamically attend to the full range of intermediate encoder states.

Slide 34: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling for tasks like attention mechanisms and machine translation. The visual example demonstrates how the network processes sequential English inputs ("I am watching television") to generate corresponding French outputs ("je regarde"), highlighting the flow of information through weight matrices (W, U, V) and biases (b1, b2) across time steps.

Slide 35: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling for tasks like attention mechanisms and machine translation. The visual example demonstrates how the network processes sequential inputs such as "I am watching television" or French phrases like "je regarde," mapping them through weight matrices (W, U, V) and biases (b1, b2) to generate outputs (y). This structure highlights the RNN's capacity to handle temporal dependencies in computational linguistics.

Slide 36: Recurrent neural network

Slide 36 introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence processing through English and French examples like "I am watching television" and "je regarde." The diagram depicts the network's internal structure with weight matrices (W, U, V) and biases (b1, b2) that process input sequences to generate outputs, highlighting the mechanism for handling temporal dependencies in computational linguistics tasks.

Slide 37: Recurrent neural network

Slide 37 introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling for tasks like machine translation. The example demonstrates how an English input sequence ("I am watching television") is processed to generate a corresponding French output sequence ("je regarde"), highlighting the network's ability to handle variable-length inputs and outputs through recurrent connections.

Slide 38: Recurrent neural network

Slide 38 introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling for tasks like machine translation. The visual example demonstrates how an RNN processes a temporal input sequence ("I am watching television") to generate a corresponding output sequence ("je regarde la télévision"), highlighting the network's ability to handle variable-length data through recurrent connections.

Slide 39: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling for tasks like machine translation and attention mechanisms. The visual example demonstrates how the network processes sequential inputs such as "I am watching television" to generate corresponding outputs like "je regarde," highlighting the flow of information through time steps with associated weights and biases.

Slide 40: Recurrent neural network

This slide introduces the Recurrent Neural Network (RNN) architecture, illustrating its application in sequence modeling and attention mechanisms for tasks like machine translation. The visual example demonstrates how an RNN processes sequential input tokens such as "I am watching television" to generate corresponding output tokens like "je regarde," highlighting the network's ability to maintain context across time steps through internal state connections.

Slide 41: Attention in machine translation

This section introduces attention in machine translation. Attention lets a decoder consult different encoder positions while generating each target token, instead of relying only on one fixed-length summary.

Slide 42: Attention

Attention computes a weighted average of representations based on their relevance to the current prediction, often repeated across multiple heads to capture distinct aspects of meaning like world knowledge. While initially introduced within RNN encoder-decoder frameworks to resolve the bottleneck problem, attention is fundamentally a general mechanism capable of replacing recurrence altogether.

Slide 43: Contextualized word embeddings

Contextualized word embeddings address the limitations of static standard embeddings by generating unique representations for each word occurrence based on its specific context, thereby capturing polysemy. Models like ELMo and BERT achieve this through successive layers where tokens are progressively adapted using information from all other tokens in the sentence. This iterative process allows even special tokens to leverage the full sentence meaning, constructing increasingly refined contextualized representations for downstream tasks.

Slide 44: ELMo

ELMo leverages language models to generate contextualized word embeddings by utilizing intermediate hidden states from RNNs, replacing static representations with dynamic ones that adapt meaning based on sentence context. Although quickly superseded by transformer models, this core concept of contextualization remains central to modern architectures. The process transforms non-contextualized inputs into representations where words like "watching" acquire specific meanings derived from their surrounding tokens, enabling more accurate predictions in task-specific neural networks.

Slide 45: ELMo

ELMo (Embeddings from Language Models) utilizes a bidirectional architecture composed of stacked Long Short-Term Memory (LSTM) networks to process text. As illustrated by the example sentence "I am watching television," the model generates context-dependent word representations by running LSTMs in both forward and backward directions, capturing information from preceding and succeeding words simultaneously.

Slide 46: ELMo

ELMo (Embeddings from Language Models) utilizes a bidirectional architecture composed of stacked Long Short-Term Memory (LSTM) networks to process text. As illustrated by the example sentence "I am watching television," the model generates context-dependent word representations by running LSTMs in both forward and backward directions, capturing information from preceding and succeeding words simultaneously.

Slide 47: ELMo

ELMo (Embeddings from Language Models) utilizes bidirectional LSTMs to generate context-sensitive word representations, as illustrated by the sentence "I am watching television." The model processes input through multiple LSTM layers, producing distinct vector outputs for each word that capture its specific meaning within the surrounding context. This architecture contrasts with static embeddings by dynamically adjusting word vectors based on their position and usage in a sequence.

Slide 48: ELMo

ELMo (Embeddings from Language Models) utilizes a bidirectional architecture composed of stacked Long Short-Term Memory (LSTM) networks to process text. As illustrated by the example sentence "I am watching television," the model generates contextualized word representations by running LSTMs in both forward and backward directions, capturing dependencies from preceding and succeeding words simultaneously.

Slide 49: Attention is all you need

The "Attention is all you need" paper introduces the Transformer architecture by radically eliminating recurrence, which in models like ELMo causes slow, sequential processing and limited context. Originally developed for machine translation, this attention-only approach has become the fundamental building block of modern Large Language Models (LLMs).

Slide 50: Transformer

The Transformer serves as the fundamental building block of Large Language Models (LLMs), functioning like an advanced feedforward network centered on a self-attention mechanism. This core architecture computes contextualized embeddings by allowing each token to attend to all other tokens in a sentence, thereby resolving word ambiguity based on surrounding context. While Transformers support bi-directional modeling, they are primarily utilized here for left-to-right language modeling, where the model predicts the next word from a probability distribution over the vocabulary using learned query, key, and value matrices.

Slide 51: Transformer

The Transformer serves as the fundamental building block of Large Language Models, functioning as an advanced feedforward network driven by attention mechanisms. It generates contextualized word embeddings by analyzing surrounding words, effectively distinguishing meanings in sentences like "The astronomer watched the star" versus "The audience watched the star." While primarily utilized for left-to-right language modeling, this section first introduces bi-directional language modeling to explain its core mechanics.

Slide 52: Transformer

The Transformer serves as the fundamental building block of Large Language Models, functioning as an advanced feedforward network driven by attention mechanisms. It generates contextualized word embeddings by analyzing surrounding words, allowing it to distinguish meanings in sentences like "the astronomer watched the star" versus "the audience watched the star." While Transformers are primarily used for left-to-right language modeling, this section first examines bi-directional language modeling to establish foundational concepts.

Slide 53: BERT

BERT is a Transformer-based NLP model introduced in 2018 that achieved state-of-the-art results by utilizing bi-directional context representations. It employs a two-stage approach: general pre-training on language modeling tasks followed by fine-tuning for specific applications, significantly outperforming previous architectures like BiLSTM and ELMo on benchmarks such as sentiment analysis and natural language inference. The model's performance scales positively with increased pre-training data and larger model sizes, though the professor notes that encoder-like models are now less dominant compared to newer large language models.

Slide 54: Bi-directional encoding

Bi-directional encoding utilizes multiple Long Short-Term Memory (LSTM) units to process sequences by capturing context from both past and future directions simultaneously. As illustrated with the sentence "I am watching television," this approach allows the model to encode the current word using information from preceding tokens as well as subsequent ones, enhancing the representation of the input sequence before generating the output.

Slide 55: Bi-directional encoding

Bi-directional encoding utilizes Long Short-Term Memory (LSTM) networks to process sequences by capturing context from both past and future directions simultaneously. As illustrated by the example sentence "I am watching television," this approach allows the model to encode each word using information from preceding tokens as well as subsequent ones, enhancing the representation of the input sequence for downstream tasks.

Slide 56: Bi-directional encoding

Bidirectional encoding reads the sequence in both directions so a token representation can use left and right context. This is suitable for encoding an input sequence, while autoregressive generation typically masks future target tokens.

Slide 57: Bi-directional encoding

Bi-directional encoding in transformers allows the model to utilize context from both preceding and succeeding tokens when predicting a masked word. As illustrated by the example "I am [MASK] watching television," the architecture processes the entire sentence simultaneously, enabling the transformer to infer the missing term based on the full surrounding linguistic environment rather than just one direction.

Slide 58: Bi-directional encoding

Bi-directional encoding in transformers involves masking approximately 15% of input words to train the model to predict them using context from both directions. For instance, in the sentence "I am [MASK] watching television," the model learns to infer the missing word by analyzing surrounding tokens simultaneously. This approach enables the architecture to capture full contextual dependencies rather than relying on unidirectional information flow.

Slide 59: Transformer

Transformers generate contextualized representations by applying multiple layers of self-attention and feed-forward computations to non-contextualized inputs, enabling efficient parallel processing instead of sequential methods. The core self-attention mechanism utilizes query and key vectors to allow tokens to adapt their meanings based on the entire input context. Successive layers progressively refine these representations, which can then be used for various NLP tasks like sentiment analysis or semantic similarity classification.

Slide 60: Transformer: self-attention

The self-attention example assigns position-dependent weights to tokens in “I am television watching.” Those weights form a weighted combination of value vectors, allowing the representation at one position to use context from other positions.

Slide 61: Transformer: self-attention

In Transformer translation tasks like "Je regarde la télévision," self-attention allows the model to attend to previously predicted tokens, such as the start symbol, using query and key vectors to track what has already been translated. Simultaneously, cross-attention enables the current token to look back at all encoder representations from the source sentence. This mechanism connects the decoder's query vector to the key vectors of contextualized source words like "Je," "regarde," and "télévision" to inform the next prediction.

Slide 62: Transformer: self-attention

The diagram illustrates self-attention for the token sequence “I am television watching.” Each token representation can draw information from positions across the input. The numbered positions and weights are part of this example; they do not by themselves define a universal attention pattern.

Slide 63: Transformer: self-attention

This animation build shows the attention weights for a selected token position. The weights indicate how strongly that position uses the other sequence positions in this example; they are not fixed linguistic rules.

Slide 64: Transformer: self-attention

The build continues the same self-attention calculation, using the token positions in “I am television watching.” The context representation is a weighted sum of information from the sequence.

Slide 65: Transformer: self-attention

This page shows another step of the attention example. The attention pattern depends on the current token and layer, so it changes with the model input.

Slide 66: Transformer: self-attention

This build continues the self-attention example. The displayed weights show how one token position distributes attention across the sequence; the resulting context-sensitive representation can incorporate information from other positions.

Slide 67: Transformer: self-attention

The final self-attention build completes the displayed weight pattern. Its function is to let each representation combine information from the sequence, not to provide a standalone explanation of the sentence.

Slide 68: BERT

The BERT block combines self-attention, a feed-forward sublayer, and residual connections. Self-attention exchanges context across token positions; the feed-forward layer transforms each position’s representation; residual additions carry the incoming representation through the block.

Slide 69: BERT

This page continues the BERT illustration. BERT uses a Transformer encoder with bidirectional context. The special tokens shown in the surrounding sequence mark input boundaries and, in classification setups, provide a sequence-level representation.

Slide 70: BERT

[CLS] and [SEP] mark sequence-level and boundary information in the BERT input format shown. BERT uses bidirectional encoder context; [CLS] is commonly used as a sequence representation for classification tasks.

Slide 71: Transformer

BERT employs the WordPiece algorithm to generate a vocabulary of subword units, balancing computational efficiency with the ability to handle rare words and neologisms by combining smaller parts. The process initializes with all characters and iteratively merges the most frequent substrings until a predefined limit is reached, ensuring that common full words retain their specific embeddings rather than being inefficiently reconstructed from individual letters. This frequency-based approach guarantees no out-of-vocabulary errors while maintaining a manageable vocabulary size for faster processing.

Slide 72: Transformer

Because Transformers process words in parallel via self-attention rather than sequentially like recurrent networks, they inherently lose information about word order. To solve this, models incorporate positional encodings: additional vectors representing a token's specific position (e.g., zero, one, two) that are summed with the initial token embeddings. This addition ensures the model retains awareness of the sequence structure despite the parallel computation.

Slide 73: BERT

BERT utilizes a pre-training and fine-tuning paradigm where the model first learns general language patterns via masked language modeling on large corpora. A special <CLS> token is prepended to inputs, with its final embedding serving as a representation of the entire sentence for downstream tasks. While this representation can be used directly, standard practice involves adding a softmax classification layer and fine-tuning all model parameters on small, labeled datasets to adapt the pre-trained knowledge to specific objectives like sentiment analysis.

Slide 74: BERT

This BERT page continues the model overview. BERT is an encoder model trained with objectives such as masked-language modeling; it can supply contextual token or sequence representations for downstream NLP tasks.

Slide 75: BERT

Slide 75 introduces the BERT model by referencing its performance results on the GLUE Multi-Task Benchmark, as established by Wang et al. in 2018. This benchmark serves as a standard evaluation framework within computational linguistics for assessing natural language understanding capabilities across diverse tasks. The slide highlights these specific results to contextualize BERT's effectiveness relative to prior models.

Slide 76: Transformer for Language Generation

While transformer encoders like BERT excel at inducing language representations through bidirectional encoding, this capability is unsuitable for generation tasks. To enable next-word prediction, transformers must be modified to attend exclusively to previous positions rather than future ones. Consequently, these models generate the most probable next token based solely on previously predicted tokens, a mechanism fundamental to all large language models.

Slide 77: Causal attention

Causal attention governs the mechanism in left-to-right causal language modeling, ensuring that when processing a specific token x_i, the model attends only to all inputs up to and including x_i. This constraint prevents the model from accessing future tokens, maintaining the directional flow required for autoregressive generation.

Slide 78: Transformer for Language Generation

Slide 78 illustrates a Transformer model performing language generation, displaying the input sequence "I am television watching" alongside special tokens like `<start>` and `[SEP]`. The output demonstrates the model's prediction of the next token in the sequence, specifically generating "am television watching . I". This example highlights how the architecture processes sequential text data to predict subsequent words within a computational linguistics context.

Slide 79: Transformer for Language Generation

Slide 79 illustrates the input sequence structure for a Transformer model performing language generation, specifically showing the tokenization of the phrase "I am television watching." The example demonstrates the inclusion of special tokens, such as `<start>` and `[SEP]`, alongside the word tokens to format the input for the model. This visual representation highlights how raw text is converted into a structured sequence of tokens required for computational processing in language generation tasks.

Slide 80: Transformer for Language Generation

This slide illustrates the input sequence structure for a Transformer model performing language generation, specifically showing how tokens like "I", "am", "television", and "watching" are combined with special markers such as `<start>` and `[SEP]`. The example demonstrates the arrangement of these elements to form a coherent input string that the model processes to predict subsequent words in the sequence.

Slide 81: Transformer for Language Generation

The diagram applies a Transformer decoder to language generation. A start token begins the output, causal masking restricts each step to earlier generated tokens, and the model predicts the next token from the current context.

Slide 82: Transformer for Language Generation

Slide 82 illustrates the application of a Transformer model for language generation using the specific input sequence "I am television watching" combined with special tokens `<start>` and `[SEP]`. The example demonstrates how the model processes this structured text to generate subsequent tokens, such as the period and the pronoun "I", within the context of computational linguistics.

Slide 83: GPT

GPT (Generative Pre-trained Transformer) models are trained on large corpora via next-word prediction, leveraging unlabelled data to acquire general language knowledge efficiently. While the fundamental architecture has remained consistent across iterations, successive versions like GPT-2 and GPT-3 have significantly increased in scale, growing from 117 million to 175 billion parameters respectively. This expansion in parameter count and training data volume represents the primary evolution of these models over recent years.

Slide 84: Conditional generation

Conditional generation produces an output sequence given conditioning information such as a prompt or encoded input. The cited framework is the setup for tasks where the model's output should depend on that context.

Slide 85: The power of scale

The power of scale in Large Language Models (LLMs) arises from applying the transformer architecture with vast numbers of parameters and extensive data. By focusing solely on the task of predicting words, these models inherently acquire the ability to solve diverse downstream tasks such as sentiment analysis, question answering, and summarization without explicit training for each specific function.

Slide 86: Examples

This slide illustrates how next-token prediction enables diverse tasks like sentiment analysis and factual retrieval. For the prompt "The sentiment of the sentence 'The new Bond movie is fantastic' is:", the model predicts the optimal label (negative, positive, or neutral) as the next token. Similarly, given "The capital of France is," the model leverages its vast training data to generate the correct answer. This mechanism also extends to summarization, where the model attends to a long input document within the initial context to produce a concise summary.

Slide 87: Summarization

This slide illustrates summarization by contrasting a detailed news story about an entrepreneur selling Boston snow with its condensed version. The original text provides extensive context regarding the $89 price, shipping restrictions to New England, and the origin of the idea during record-breaking snowfall. In contrast, the summary extracts only the core facts: the product details, quantity, and specific geographic limitations, demonstrating how summarization reduces length while preserving essential information.

Slide 88: Three architectures for LLMs

Large language models utilize three primary architectures: the Encoder (e.g., BERT), which predicts masked words using bidirectional context and is typically fine-tuned with supervised data; the Decoder (e.g., GPT), which performs word prediction for tasks framed as sequential generation; and the Encoder-Decoder (e.g., T5), designed to map one sequence to another for applications like machine translation and speech recognition.

Slide 89: Encoder-decoder

The encoder-decoder architecture is the original Transformer design developed for machine translation and other sequence-to-sequence tasks. The encoder constructs bi-directional contextualized representations of the source input, while the decoder generates the target sequence left-to-right using self-attention on previous outputs and cross-attention to attend to the encoded source. This framework allows diverse NLP problems, such as speech recognition or text reconstruction in models like T5, to be reframed as mapping one sequence to another.

Slide 90: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input "je regarde la télévision" is processed to generate the output sequence "<start> I am television watching". This slide demonstrates how the model maps source language tokens to target language tokens, including the use of a special start token to initiate the generation process.

Slide 91: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input "je regarde la télévision" is processed to generate the output sequence "<start> I am television watching". This slide demonstrates how the model maps source language tokens to target language tokens, including the use of a special start token to initiate generation. The example highlights the sequential nature of translating entire sentences within computational linguistics.

Slide 92: Encoder-decoder

The encoder-decoder architecture processes input sequences, such as the French phrase "je regarde la télévision," to generate corresponding output sequences in a target language, like "I am watching television." This example illustrates how the model maps source tokens to target tokens while utilizing special markers, such as `<start>`, to initiate the generation process.

Slide 93: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input sequence "je regarde la télévision" is processed to generate the output "I am watching television." The model incorporates a special `<start>` token at the beginning of the generation process to initiate the decoding sequence. This slide demonstrates how the system transforms an encoded source representation into a target language sentence, highlighting the sequential nature of the output generation in computational linguistics tasks.

Slide 94: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input "je regarde la télévision" is processed to generate the output sequence "<start> I am television watching". This demonstration highlights how the model maps source language tokens to target language tokens, including the use of a special start token to initiate the generation process.

Slide 95: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input sequence "je regarde télévision la" is processed to generate the output "I am television watching." The model incorporates a special `<start>` token to initiate the decoding process, demonstrating how sequential data is transformed from one language representation to another within computational linguistics.

Slide 96: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example, where the input sequence "je regarde la télévision" is processed to generate the output "I am watching television." The model incorporates a special `<start>` token at the beginning of the generation process to initiate the decoding sequence. This slide demonstrates how the system maps an entire source sentence into a target sentence through this sequential mechanism.

Slide 97: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input sequence "je regarde télévision la" is processed to generate the output "I am television watching," preceded by a special start token. This slide demonstrates how the model maps source language tokens to target language tokens sequentially, highlighting the fundamental mechanism of translating entire sentences rather than individual words in isolation.

Slide 98: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input sequence "je regarde télévision la" is processed to generate the output "I am television watching," preceded by a special start token. This demonstration highlights how the model maps source language tokens to target language tokens sequentially, forming the basis for sequence-to-sequence tasks in computational linguistics.

Slide 99: Encoder-decoder

The encoder-decoder architecture is illustrated by translating the French phrase "je regarde la télévision" into the English sentence "I am watching television." This example demonstrates how the model processes input tokens and generates output sequences, utilizing a special `<start>` token to initiate the decoding process.

Slide 100: Encoder-decoder

The encoder-decoder architecture is illustrated using a French-to-English translation example where the input "je regarde la télévision" is processed to generate the output sequence "**<start>** I am television watching". This demonstration highlights how the model maps source language tokens to target language tokens, including the use of a special start token to initiate generation. The slide serves as a concrete visualization of the sequential mapping process fundamental to encoder-decoder systems in computational linguistics.

Slide 101: Encoder-decoder

The encoder-decoder architecture processes input sequences, such as the French phrase "je regarde la télévision," to generate corresponding output sequences like the English translation "I am watching television." This model incorporates a special start token, denoted as `<start>`, to initialize the decoding process. The example illustrates how the system transforms source language tokens into target language tokens within computational linguistics applications.

Slide 102: Encoder-decoder

The encoder-decoder architecture processes input sequences, such as the French phrase "je regarde la télévision," to generate corresponding output sequences like the English translation "I am watching television." The model incorporates a special `<start>` token to initiate the decoding process and manages word order rearrangements, transforming the literal sequence into a grammatically correct target sentence.

Slide 103: Encoder-decoder

The encoder-decoder architecture processes input sequences, such as the French phrase "je regarde la télévision," to generate corresponding output sequences like "I am watching television." This example illustrates how the model handles tokenization and sequence alignment, including the use of special tokens like `<start>` and punctuation to structure the translation task within computational linguistics.

Slide 104: Encoder-decoder

The encoder-decoder architecture translates the French input "je regarde la télévision" into English by first generating a literal word-for-word sequence ("I am television watching") and then reordering it to produce the grammatically correct output "am watching television." This process is bounded by special start and stop tokens, illustrating how the model handles both translation and syntactic restructuring within a single generation framework.

Slide 105: T5

Slide 105 introduces T5, referencing the seminal 2020 paper by Raffel et al. titled "Exploring the limits of transfer learning with a unified text-to-text transformer." This model represents a key development in computational linguistics where diverse natural language processing tasks are framed as a single text-to-text problem to maximize transfer learning capabilities.

Slide 106: T5

Slide 106 introduces T5, referencing the seminal 2020 paper by Raffel et al. titled "Exploring the limits of transfer learning with a unified text-to-text transformer." This model represents a key development in computational linguistics where diverse natural language processing tasks are framed as a single text-to-text problem to maximize transfer learning capabilities. The slide serves as a citation marker for this foundational work within the course context.

Key points to remember

The core of the lecture focuses on the Transformer architecture, which utilizes self-attention to allow tokens to attend to all other tokens simultaneously, capturing bidirectional context and resolving polysemy more effectively than static embeddings or RNNs. Specific implementations like BERT leverage this through masked language modeling, where the model predicts missing tokens using surrounding context, followed by fine-tuning for specific tasks using a special <CLS> token for sentence representation. In contrast, generative models like GPT employ causal attention to restrict access to future tokens, enabling autoregressive next-token prediction. The lecture concludes by emphasizing the power of scale, noting that training these architectures on vast corpora for the single objective of word prediction allows them to implicitly master diverse downstream tasks such as sentiment analysis, question answering, and summarization without explicit task-specific training.